JETBRAINS | Kotlin

Testing

kotlinglang.org/education



Testing

The software development experience we've accumulated over the many years unfortunately tells us that

- Every program contains bugs.
- If a program does not contain bugs, the algorithm that it implements contains them.
- If neither the program nor the algorithm contains bugs, no one needs the program (almost always).

Testing

However, everything is relative:

- A bug in a website might not really hurt anybody
- A bug in rocket launch calculations can result in terrible consequences and expenses (Ariane 5)
- A bug in a radiation-treatment device can lead to deaths (Therac-25)



Ariane 5

Testing: history

- 60s: Do exhaustive testing.
- Early 70s: Show that the program works correctly.
- Late 70s: Show that the program does not work correctly.
- 80s: Prevent defects throughout development.

Testing: goals

- Correct behavior of the product in all conditions.
- Compliance with the requirements.
- Information about the current state of the product.
- Error prevention and detection.
- Development cost reduction.

Testing: principles

- Testing demonstrates the presence of defects, but it does not prove their absence.
- The earlier the better.
- The absence of bugs is not an absolute goal.
- Many more.

Testing: types

- Functional – Checking the behavior given in the specifications.

Testing: types

- Functional – Checking the behavior given in the specifications.
- Load – Simulating a real load (for example, a certain number of users on the server).

Testing: types

- Functional – Checking the behavior given in the specifications.
- Load – Simulating a real load.
- Stress – Checking the system's operation under abnormal conditions (for example, a power outage or a huge number of operations per second).

Testing: types

- Functional – Checking the behavior given in the specifications.
- Load – Simulating a real load.
- Stress – Checking the system’s operation under abnormal conditions.
- Configuration – Checking software using different system configurations.

Testing: types

- Functional – Checking the behavior given in the specifications.
- Load – Simulating a real load.
- Stress – Checking the system’s operation under abnormal conditions.
- Configuration – checking software using different system configurations.
- Regression – Making sure new changes did not break anything that had worked previously.

Testing: types

- Functional – Checking the behavior given in the specifications.
- Load – Simulating a real load.
- Stress – Checking the system's operation under abnormal conditions.
- Configuration – Checking software using different system configurations.
- Regression – Making sure new changes did not break anything that had worked previously.

Others, e.g. security, compliance, etc.

Testing: levels

- Unit testing – Testing components separately (checking modules, classes, functions).

Testing: levels

- Unit testing – Testing components separately (checking modules, classes, functions).
- Integration testing – Checking the interaction of components and program modules.

Testing: levels

- Unit testing – Testing components separately (checking modules, classes, functions).
- Integration testing – Checking the interaction of components and program modules.
- System testing – Checking the entire system.

Testing: levels

- Unit testing – Testing components separately (checking modules, classes, functions).
- Integration testing – Checking the interaction of components and program modules.
- System testing – Checking the entire system.
- Acceptance testing – Verifying system compliance with all the client requirements.

Unit testing

For each non-trivial function, their own tests are written that check that the method works correctly:

- Frequent launch expected ⇒ should run fast
- One test ⇒ one use case

Unit testing in Kotlin

The JUnit5 framework is the most popular way to test Java and Kotlin programs.

```
dependencies {
      ...
      testImplementation(platform("org.junit:junit-bom:
5.8.2"))
      testImplementation("org.junit.jupiter:junit-
jupiter:5.8.2")
}

tasks.test {
   useJUnitPlatform()
}
```

To run tests: `./gradlew test`

Unit testing in Kotlin

```
class MyTests {
     @Test
     @DisplayName("Check if the calculator works
correctly")
     fun testCalculator() {
           Assertions.assertEquals(
                3,
                myCalculator(1, 2, "+"),
                "Assertion error message"
           )
     }
}
```

Unit testing in Kotlin

```
class MyParametrizedTests {
     companion object {
          @JvmStatic
          fun calculatorInputs() = listOf(
               Arguments.of(1, 2, "+", 3),
               Arguments.of(0, 5, "+", 5),
          )
     }
     @ParameterizedTest
     @MethodSource("calculatorInputs")
     fun testCalculator(a: Int, b: Int, op: String,
expected: Int) {
          Assertions.assertEquals(expected,
myCalculator(a, b, op), "Assertion error message")
     }
}
```

Unit testing in Kotlin

There are many annotations for tests customization.

- `@BeforeEach` – Methods with this annotation are run before each test.
- `@AfterEach` – Methods with this annotation are run after each test.
- `@BeforeAll` – Methods with this annotation are run before all tests in the class.
- `@AfterAll` – Methods with this annotation are run after all tests in the class.

Code quality

Often, testing includes checking not only the correctness of the functionality but also the **quality** of the code itself.

Static code analyzers such as detekt, ktlint, and diktat exist to help you avoid having to do this manually.

The build of static analyzers should also be green.

JETBRAINS | Kotlin

Thank you

